


Analiza Algoritmilor

# Algorithms & Complexity Theory

Matei Popovici

## Mergesort

Split the array in half
Make copies of each half

Recursively sort the array

## Quicksort

Choose some value x from the array
Reorder the array into values ≤x and ≥x.

Recursively sort in place.

# Quicksort partitioning




```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```

# Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```




**A**

**pivot = 5**

# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




A

i

j

pivot = 5

# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




A

i

j

pivot = 5

# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



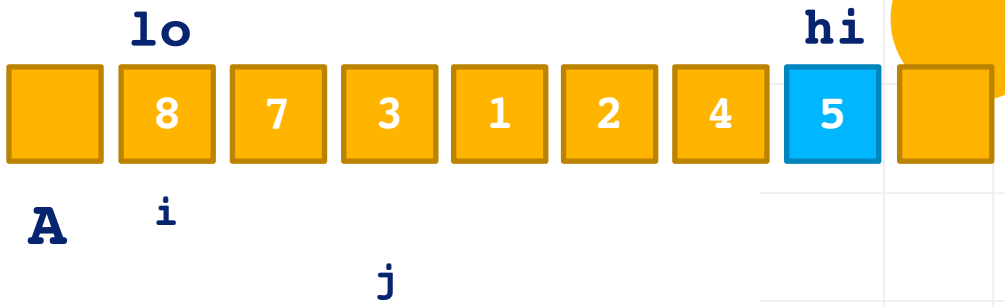

A i

j

pivot = 5

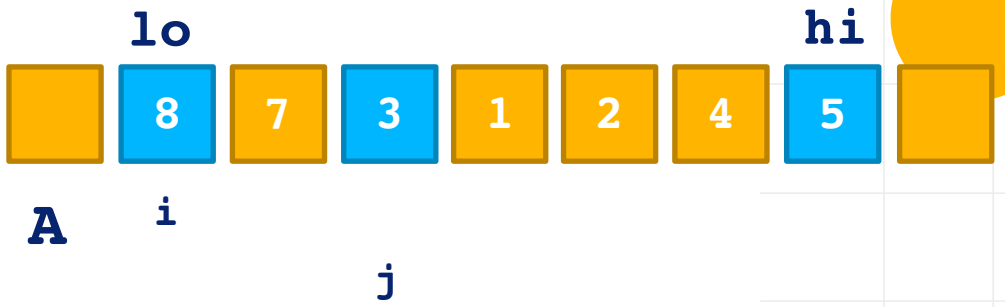
# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



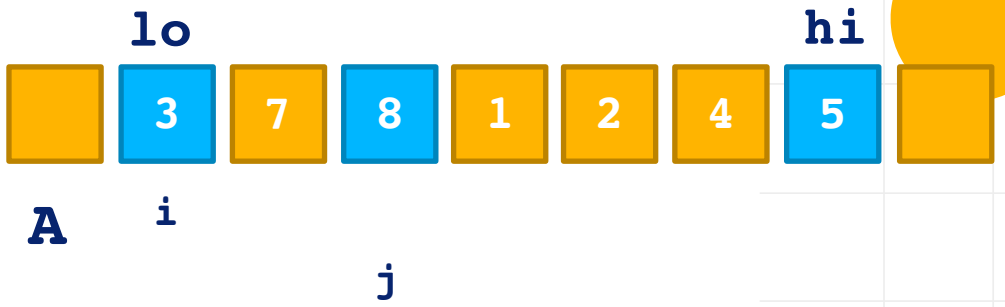
# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



# Quicksort partitioning

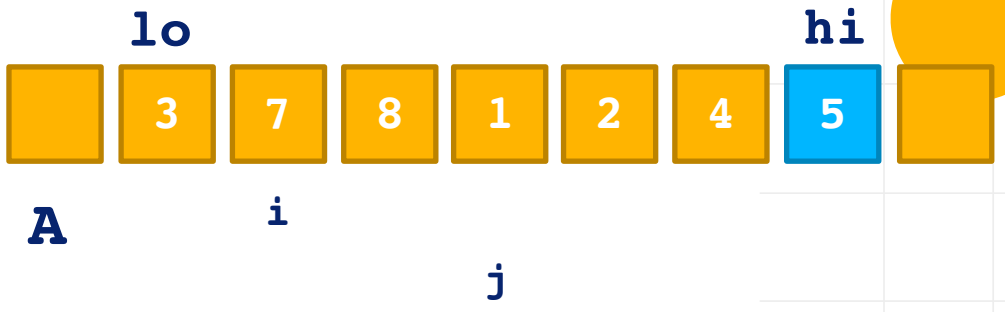
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




pivot = 5

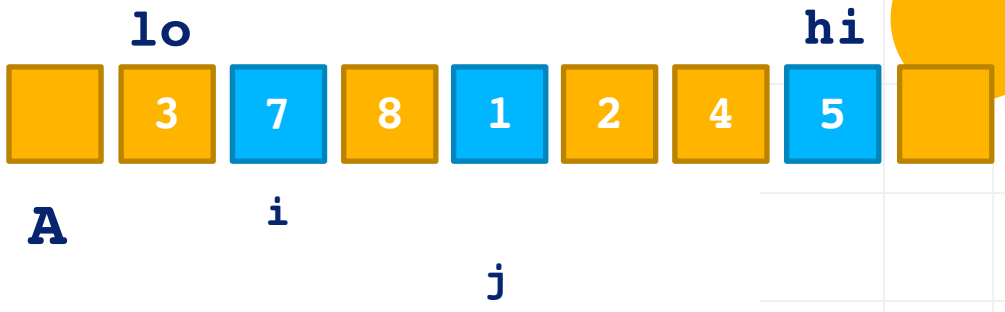
# Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



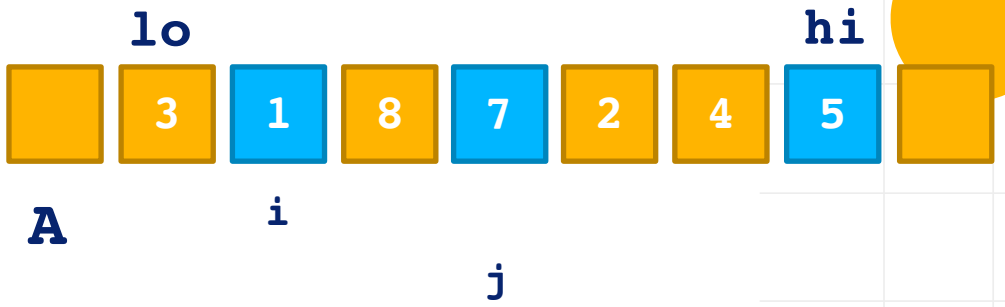
# Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



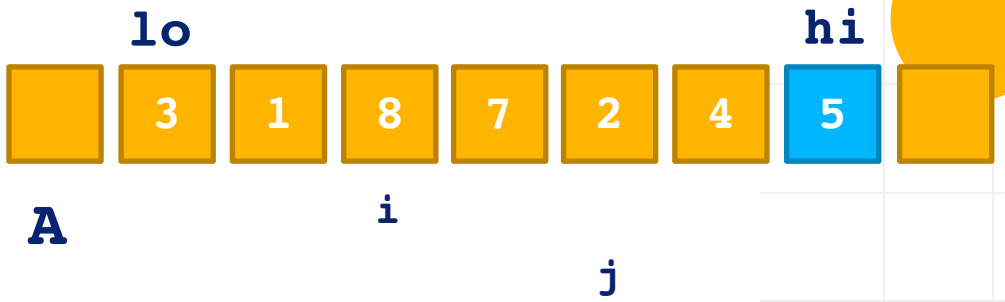
# Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



# Quicksort partitioning

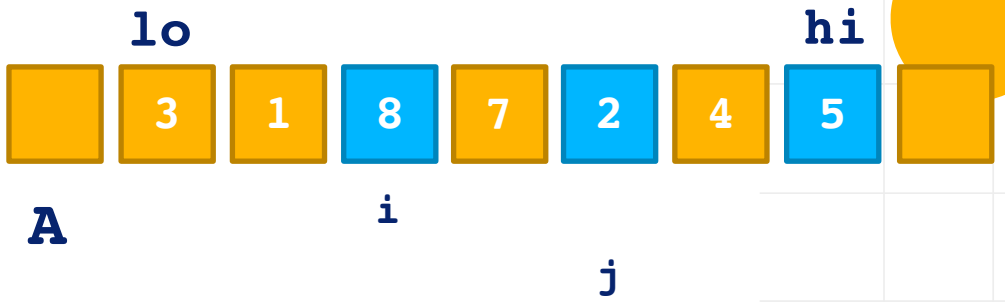
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




pivot = 5

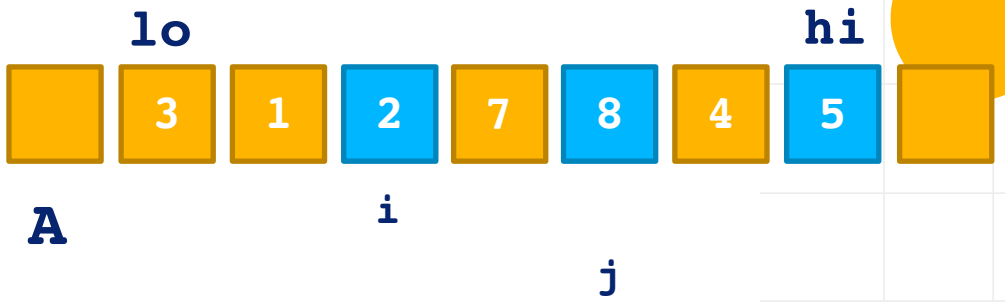
# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



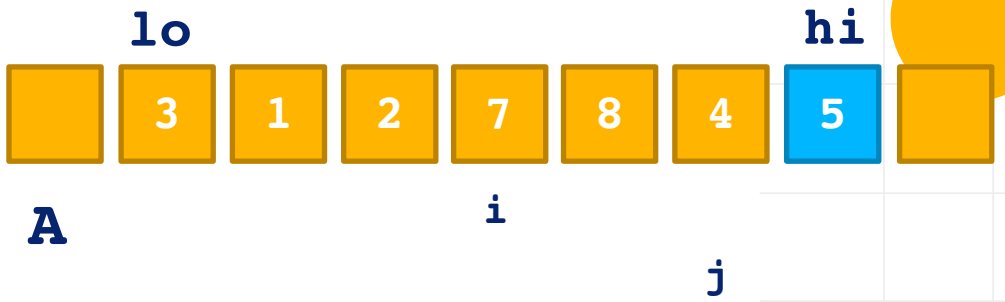
# Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



# Quicksort partitioning

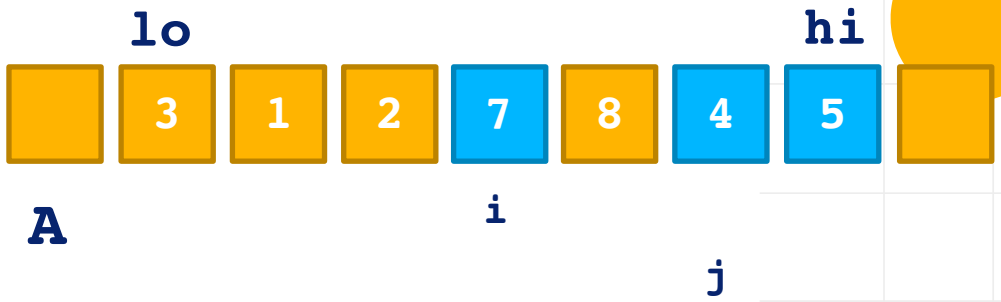
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




pivot = 5

# Quicksort partitioning

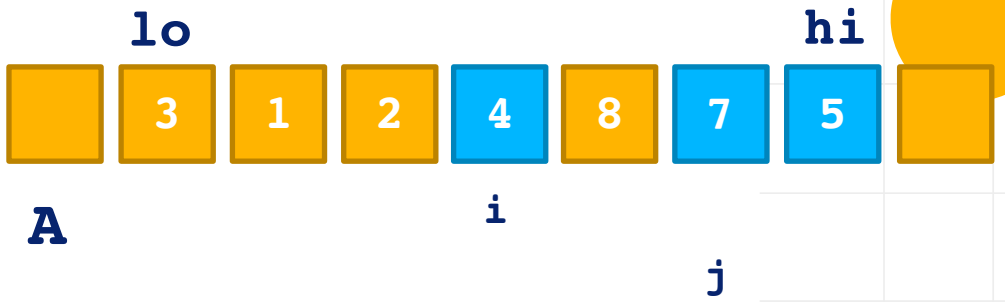
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




pivot = 5

# Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




pivot = 5

# Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```




pivot = 5

# Quicksort

```
algorithm quicksort(A, lo, hi)
    if lo < hi then
        p := partition(A, lo, hi)
        quicksort(A, lo, p - 1)
        quicksort(A, p + 1, hi)
```

# Complexity analysis

## Worst-case

Pivot is the largest element

$$T(n) = T(n-1) + \Theta(n)$$

$$T(n) = \Theta(n^2)$$

## Best-case

The array is split in half

$$T(n) = 2T(n/2) + \Theta(n)$$

$$T(n) = \Theta(n\log(n))$$

# Quicksort

```
algorithm quicksort(A, lo, hi)
    if lo < hi then
        p := partition(A, lo, hi)
        quicksort(A, lo, p - 1)
        quicksort(A, p + 1, hi)
```

# Complexity analysis

## Average-case

The array is split at 9/10 of its size

$$T(n) = T(n/10) + T(9n/10) + \Theta(n)$$

$$T(n) = \Theta(n\log(n))$$

# Probabilistic analysis of Quicksort

# Bike universe



I go to UPB by bike in a sunny day



I go to UPB on foot in a rainy day



I go to UPB by bike in a rainy day



I go to UPB by bike in a sunny day and I get a flat tire

# Dice universe



A two-dice roll
36 different outcomes

# Random variables

$$X : U \longrightarrow \mathbb{R}$$

**U – universe**

a set of events that might take place

a set of outcomes resulting from an experiment

**X(e) – a value associated to event e**

a metric

a measurement

# Bike universe

| | D<br>how much time does it take to go to UPB |
|---|---|
|  I go to UPB by bike in a sunny day | 12 min |
|  I go to UPB on foot in a rainy day | 25 min |
|  I go to UPB by bike in a rainy day | 16 min |
|  I go to UPB by bike in a sunny day and I get a flat tire | 40 min |

# Dice universe



$S(e)$ = the sum of the dice faces

# Probability distribution

$$P : U \longrightarrow [0,1]$$

$P(e)$ – the probability that event $e$ takes place

# Bike universe

P



I go to UPB by bike in a sunny day

60%



I go to UPB on foot in a rainy day

24%



I go to UPB by bike in a rainy day

15%



I go to UPB by bike in a sunny day and I get a flat tire

1%

# Dice universe



$P(e) = 1/36$

# Expectation of a random variable

$$\mathbb{E}[X] = \sum P(e) * X(e)$$

Repeat an experiment very many times
Observe a process for a long time

Record the values $X(e)$ that you observe

Compute the average value of $X$, over all observed events $e$

If you repeat the experiment long-enough, each event $e$ will occur with probability $P(e)$

# Bike universe:

$\mathbb{E}[D]$ - how long does a trip to UPB last, on average?

| | D | P | |
|---|---|---|---|
|  I go to UPB by bike in a sunny day | 12 min | 60% | =7,2 |
|  I go to UPB c… rainy day | | …% | =6 |
|  I go to UPB… rainy day | | 15% | =2,4 |
|  I go to UPB by bike in a sunny day and I get a flat tire | 40 min | 1% | =0,4 |

$\mathbb{E}[D] = 16min$

# Dice universe

$\mathbb{E}[S]$- what is the average sum of dice-faces?



# Dice universe



$\mathbb{E}[S]$ - what is the average sum of dice-faces?

The smart way is to use ***linearity of expectation***

$$\mathbb{E}[S]=\mathbb{E}[S_1+S_2]$$

Random variables for the sums of each face

# Dice universe



$\mathbb{E}[S]$ - what is the average sum of dice-faces?

The smart way is to use ***linearity of expectation***

$$\mathbb{E}[S]=\mathbb{E}[S_1+S_2] = \mathbb{E}[S_1] + \mathbb{E}[S_2]$$

$$\mathbb{E}[S_i]= (1+2+3+4+5+6)/6 = 3,5$$

$$\mathbb{E}[S]= 3,5 + 3,5 = 7$$

# Quicksort universe

Assume for simplicity that elements in the array are unique

Event $e_i$ means that the $i$–th element is the pivot

Probability $P(e_i) = 1/n$ where $n$ is the size of the array

Random variable $Q_n$ where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

# Quicksort universe

Random variable $Q_n$ where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + T(i-1) + T(n-i)$$

# Quicksort universe

Random variable $Q_n$ where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + T(i-1) + T(n-i)$$

The ***expected*** execution time of Quicksort

# Quicksort universe

Random variable $Q_n$ where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + \mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i})$$

The ***expected*** execution time of Quicksort

# Quicksort universe

Random variable $Q_n$ where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + \mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i})$$

$$\mathbb{E}(Q_n) = \Theta(n) + 1/n * \Sigma\ (\mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i}))$$

# Quicksort universe

Random variable $Q_n$ where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + \mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i})$$

$$T(n) = \Theta(n) + 1/n * \Sigma\ (T(i-1) + T(n-i))$$

# Quicksort universe

$$T(n) = \Theta(n) + 1/n * \Sigma (T(i-1) + T(n-i))$$

# Quicksort universe – algebrical part

$$T(n) = \Theta(n) + 1/n * \sum_{0<i\leq n} (T(i-1) + T(n-i))$$

$$T(n) = \Theta(n) + 2/n * \sum_{0\leq i<n} T(i)$$

# Quicksort universe – algebrical part

$$T(n) = \Theta(n) + 1/n * \sum_{0<i\leq n} (T(i-1) + T(n-i))$$

$$T(n) = n + 2/n * \sum_{0\leq i<n} T(i)$$

$$nT(n) = n^2 + 2 * \sum_{0\leq i<n} T(i)$$

$$(n-1)T(n-1) = (n-1)^2 + 2 * \sum_{0\leq i<n-1} T(i)$$

# Quicksort universe – algebrical part

$$nT(n) = n^2 + 2 * \sum_{0 \le i < n} T(i)$$

$$(n-1)T(n-1) = (n-1)^2 + 2 * \sum_{0 \le i < n-1} T(i)$$

$$nT(n) - (n-1)T(n-1) = n^2 - (n-1)^2 + 2T(n-1)$$

$$nT(n) = 2n-1 + (n+1)T(n-1)$$

$$nT(n) \le 2(n+1) + (n+1)T(n-1)$$

# Quicksort universe – algebrical part

$$nT(n) \leq 2(n+1) + (n+1)T(n-1)$$

$$T(n)/(n+1) \leq 2/n + T(n-1)/n \qquad F(n) = T(n)/(n+1)$$

$$F(n) \leq 2/n + F(n-1)$$

$$F(n) \leq 2(1/n + 1/(n-1) + \ldots + 1/2 + 1)$$

$$F(n) \leq 2H_n \simeq 2\lg(n)$$

$$T(n) \leq 2(n+1)\lg(n) \qquad T(n)=O(n \log(n))$$